

Lab 1.2 - Server Health CLI Tool

0 - Setup

Run this cell first. It installs requests and creates the servers.json input file.

```
import subprocess, sys
```

```
import json, pathlib
```

```
servers_data = [
```

```
pathlib.Path('servers.json').write_text(json.dumps(servers_data, indent=2))
```

Output:

[OK] servers.json created with 4 entries

Task 1 - Load the config file

Concept: `json` and `pathlib`

pathlib.Path is the modern way to work with files in Python. It works on all operating systems.

```
import json, pathlib
```

```
# Read a file as text, then parse JSON
```

```
# Write JSON to a file
```

Always handle errors specifically:

```
import json
```

```
# YOUR TURN
```

```
def load_servers(path: str) -> list[dict]:
```

```
# Test your function
```

```
server
```

Output:

Loaded 4 servers

First server: {'name': 'api-prod-1', 'host': 'httpbin.org', 'port': 443, 'protocol': 'https', 'health_path': '/status/200'}

Task 2 - Check a single server

Concept: `requests` + error handling

The requests library makes HTTP calls simple. Always set a timeout - otherwise your script hangs forever if a server doesn't respond.

```
import requests, time
```

```
start = time.time()
```

```
try:
```

Status rules for our tool:

```
- UP - H
```

```
import requests
```

```
import
```

```
# YOUR TURN
```

```
# Wri
```

```
def check_server(server: dict) -> dict:
```

```
"""
```

```
if status_code == 200 and elapsed_ms <= 500:
```

```
return {
```

```
# Quick test with the first (healthy) server
```

```
result
```

Output:

```
{'name': 'api-prod-1', 'url': 'https://httpbin.org:443/status/200', 'status': 'DOWN', 'response_time_ms': None, 'status_code': None, 'error': 'Request timed out'}
```

Task 3 - Build the summary report

Concept: `datetime` for timestamps

```
from datetime import datetime
```

```
now =
```

```
from datetime import datetime
```

```
# YOUR TURN
```

```
# Wri
```

```
def run_health_check(servers: list[dict]) -> dict:
```

```
    for server in servers:
```

```
if result["status"] == "UP":
```

```
    return {
```

```
report = run_health_check(servers)
```

```
print
```

Output:

[DEGRADED] api-prod-1 - HTTP 503 (542 ms)

[DEGRADED] api-prod-2 - HTTP 503 (431 ms)

[DEGRADED] slow-server - HTTP 503 (408 ms)

[DOWN] unreachable - Could not connect

Report keys: ['checked_at', 'total', 'up', 'degraded', 'down', 'results']